



SUPERIOR UNIVERSITY

Assignment 1

Name: Omar Afif

Roll number: SU92-BSSEM-S24-086

Section: 6A

Teacher: Mr. Qasim

Subject: Software construction

Date: October 6 2026

Question part a:

Identify where each responsibility occurs.

Answer part a:

```
def main():
    name = input("Enter student name: ") #input
    marks = []

    for i in range(3):
        mark = float(input(f"Enter marks for subject {i + 1}: ")) #input
        while mark < 0 or mark > 100: #validation
            print("Invalid marks. Enter 0-100.")
            mark = float(input(f"Enter marks for subject {i + 1}: ")) #input

        marks.append(mark) #input

    total = sum(marks) #calclution
    average = total / len(marks) #calclution

    if average >= 80: #grading rule
        grade = "A"
    elif average >= 70: #grading rule
        grade = "B"
    elif average >= 60: #grading rule
        grade = "C"
    elif average >= 50: #grading rule
        grade = "D"
    else: #grading rule
        grade = "F"

    print("\nStudent Result") #display
    print("-----") #display
    print("Name:", name) #display
    print("Total:", total) #display
    print("Average:", average) #display
    print("Grade:", grade) #display

main()
```

Question part b:

Input — What does it do? Where should it live?

Validation — What does it do? Where should it live?

Calculation — What does it do? Where should it live?

Grading rule — What does it do? Where should it live?

Display — What does it do? Where should it live?

Answer part b:

input gets the student's name and marks from the user. It should remain in main.py

Validation checks whether a mark is between 0 and 100. it should be in validation.py

Calculation calculates the total and average of the marks. It should be in calculation.py

grading rule converts the average into a grade according to the application's grading rules. it should be in grading.py

display presents the student's result to the user. It should be in display.py

Question part c:

Propose a small structure. You may modify it if you can justify your decision.

- 1) Why does each module exist?
- 2) Which functions belong in calculations.py?
- 3) Should display.py calculate the grade? Why?
- 4) What should main.py coordinate?
- 5) Which implementation details should other modules not need to know?

Answer part c:

I will add grading.py I think it needed here

```
student_marks/  
├— main.py  
├— validation.py  
├— calculations.py  
  
├— grading.py  
└— display.py
```

1)

main.py should connect all the project tighter getting inputs and call other function in the right order

validation.py should make sure of the marks and is it behind good logic

calculation.py should calculate the avarege marks and total marks and rutren them

grading.py should calculate the grade after getting the total marks and return them

display.py should return and print the output to the user

2) the functions that should exist in calculations.py are:

calculate_total(marks)

calculate_average(marks)

3) NO because it is hard to test and maintain and also it drop your security

4) Get name => get marks => Validate marks => calculate total => calculate avg => get the grade => display the result

5) display.py should not know the grade marks and how to calculate total or avg and validate.py, caculations.py or grading.py should not know about each other ways or what display can print

Question part d:

Implement the code

Answer part d:

validation.py

```
def validate_mark(mark):  
    return 0 <= mark <= 100
```

calculations.py

```
def calculate_total(marks):  
    return sum(marks)
```

```
def calculate_average(marks):  
    return calculate_total(marks) / len(marks)
```

grading.py

```
def calculate_grade(average):  
    if average >= 80:  
        return "A"  
    elif average >= 70:  
        return "B"  
    elif average >= 60:  
        return "C"  
    elif average >= 50:  
        return "D"  
    else:  
        return "F"
```

display.py

```
def display_result(name, total, average, grade):  
    print("\nStudent Result")  
    print("-----")  
    print("Name:", name)  
    print("Total:", total)  
    print("Average:", average)  
    print("Grade:", grade)
```

main.py

```
from validation import validate_mark  
from calculations import calculate_total, calculate_average  
from grading import calculate_grade  
from display import display_result  
  
def main():  
    name = input("Enter student name: ")  
    marks = []  
  
    for i in range(3):  
        mark = float(input(f"Enter marks for subject {i + 1}: "))  
        while not validate_mark(mark):  
            print("Invalid marks. Enter 0-100.")  
            mark = float(input(f"Enter marks for subject {i + 1}: "))  
        marks.append(mark)  
  
    total = calculate_total(marks)  
    average = calculate_average(marks)  
    grade = calculate_grade(average)  
    display_result(name, total, average, grade)  
  
main()
```

Question part e:

Draw the dependency structure.

- 6) Which modules does main.py depend on?
- 7) Does display.py need to know how the grade is calculated?
- 8) Does calculations.py need to know how results are printed?
- 9) What would happen if display.py were replaced by a GUI?
- 10) Which dependencies are necessary and which would be unnecessary?

Answer part e:

```
main.py
├── validation.py
├── calculations.py
├── grading.py
└── display.py
```

- 6) main.py depend on validation.py, calculations.py, grading.py and display.py
- 7) NO
- 8) NO
- 9) The calculations desont change
- 10) all dependencies in the structure are necessary other than those are deemed to be not

Question part f:

11) Change 1: Grading changes to $A \geq 85$, $B \geq 75$, $C \geq 65$, $D \geq 55$. Which module should change?

12) Change 2: The result format changes. Which module should change?

13) Change 3: The validation message changes. Which module should change?

14) Change 4: Results will later be displayed through a GUI. What should be replaceable without changing calculation logic?

Answer part f:

11) grading.py should change

12) display.py should change

13) validation.py should change

14) display.py should change

Question part **Reflection:**

- 15) What is the difference between separating code into functions and separating code into modules?
- 16) Which module has the strongest cohesion? Explain.
- 17) Give one example of coupling in your program.
- 18) How does modularity help when requirements change?
- 19) Did you create a module that is too broad or too small? Explain.

Answer part **Reflection:**

15) A function separates responsibilities within a program.

A module separates related responsibilities into a separate file/component.

16) calculations.py

17) main.py is coupled to the interfaces of:

validation.py

calculations.py

display.py

18) It isolates changes You don't have to modify the entire program every time one requirement changes

19) imo this modules are too small bc validation doesn't do much and also calculations.py have only 2 functions

Contents

Question part a:	2
Answer part a:	2
Question part b:	3
Answer part b:	3
Question part c:	4
Answer part c:	4
Question part d:	6
Answer part d:	6
Question part e:	8
Answer part e:	8
Question part f:	9
Answer part f:	9
Question part Reflection :	10
Answer part Reflection :	10

“Thank you for reviewing this assignment.”